

Computational Genomics: Sequences

HW2 Written

Ashrith Kollu

1.
 - a) The base call would be C since it's the strongest signal. 6/9 total units indicate C, so the probability that the call is wrong is 1/3. $Q = -10\log(1/3) = 4.77 \approx 5$. Using Phred +33 encoding, we would use ASCII character $5+33 = 38$, which corresponds to &.
 - b) I corresponds to 73 and + corresponds to 43. First base: $Q = 73-33 = 40$. $P = 1/10000$ Last base: $Q = 43-33 = 10$. $P = 1/10$ Illumina sequencing creates clusters and adds bases to the cluster one at a time per cycle. Since this depends on the whole cluster growing together, mistakes during the sequencing cycles could result in some of the strands in the cluster becoming out of sync with the rest of the cluster. This is increasingly probable as the clusters undergo more and more cycles, which is why later bases will generally have worse quality.
 - c) Dataset B has the aberrant cycle at cycle number 38. The program counted 6041 low-quality bases there.
2.
 - a) Given ATGCATGCATGC, we have the kmers ATGC, TGCA, GCAT, CATG, ATGC, TGCA, GCAT, CATG, and ATGC. ATGC's reverse complement (RC) is GCAT, so we keep the former. TGCA's RC is TGCA, so we keep the former. GCAT's RC is ATGC, so we keep the latter. Continuing, we keep CATG, ATGC, TGCA, ATGC, TGCA, ATGC, CATG, and ATGC. This gives us distinct kmers ATGC, TGCA, and CATG. Thus, we keep 3 distinct canonical kmers.
 - b) We have ATGCA, TGCAT, GCATG, CATGC, ATGCA, TGCAT, GCATG, CATGC. Only ATGCA, TGCAT, GCATG, CATGC are unique. The canonical kmers are ATGCA, ATGCA, CATGC, and CATGC. Thus, only 2 distinct canonical kmers exist for this sequence at $k=5$.
3.
 - a) One reason the read might not match exactly is because the donor might have minor differences in their DNA, like indels or single nucleotide variants. Though DNA from person-to-person is nearly identical, there are differences scattered throughout the genome. Thus, the sequences might not be exact matches even though they came from the same location on the genome. Another reason an exact match could fail is if there are errors in the reads. Sequencing isn't perfect, and wrong base calls can occur. This is especially true in repetitive areas in the genome. Mistakes in the reads could prevent an exact match from being found even though that specific segment of the genome is the same between the donor and the reference.
 - b) The approaches I used in Q3 and Q4 are both offline because I built a kmer dictionary using the reference and then queried it with data from reads. The building of the kmer dictionary is a form of preprocessing the reference sequence. For q3, after preprocessing the reference, I used a 'seed and extend' algorithm to split the reads into 4 partitions and searched for exact matches to 'seed' an alignment and then verify that alignment – the 'extend' phase. For q4, I used a similar approach except this time, I split into two partitions and allowed for the read frame to vary by 1 base pair. Allowing the frame to vary allowed for the capture of alignments with indels while still making efficient use of the 'seed and extend' algorithm.
4.
 - a) The first mismatch occurs at offset 3, and the text character there is a G. Bad character rule skips until there is a match or the pattern moves past that character. Since there is no G in P, the rule will skip 3 alignments to move P past that offset position. For the good suffix rule, the alignments will skip until the substring of T that matched appears elsewhere in P or until P moves past the

substring. Here, TC matched at offsets 4 and 5, but G did not at offset 3. The next occurrence of the substring is the TC at P offsets 2 and 3, so this is shifted into the offset 5 position, thus skipping 1 alignment.

- b) The first mismatch occurs at offset 2, and the text character there is a A. Bad character rule skips until there is a match or the pattern moves past that character. Since there is an A in the very next alignment at offset 1 of the pattern, no alignments are skipped. With the good suffix rule, no other occurrences of CTC exist in the pattern. However, there is a prefix of P in the suffix of the matched substring of T. Thus, 4 alignments are skipped to move the C at the beginning of P to offset 5 of the reference.
5. The reads came from the Evo phage genome. I used my code from q4 to compare the reference sequences with the mystery reads. After the corresponding files were generated, I wrote a short script that essentially tallied up the number of lines that reported an edit distance within 1. It showed that the Evo file reported 30 more matches than the Phix file, which means that it must be a closer match. With an error rate of about 2%, the probability of a read having no errors is 0.98^{20} and for 1 error it is $\binom{20}{1} * 0.98^{19} * 0.02$. Adding these up, I expect about 94% of reads to be within 1 edit distance, which is about 188 out of the 200 reads. Evo is the closest to that, with 191 matches as opposed to Phix's 160, reinforcing this conclusion.
6. a) AGCTAG
- b) AAATGC. For good character rule, the mismatch occurs with the reference A at offset 2. Since there is no other A in P, it shifts until P moves past that offset. This corresponds to a shift of 3, which skips 2 alignments. For the good suffix rule, T, G and C match at offsets 3, 4, 5. The next occurrence of this substring is at offsets 0, 1, 2 of the pattern, so two alignments are skipped.
 - c) There are 16 alignments. Since the pattern p has a sequence that repeats, TGC, every 3 base pairs, any mismatch in offset positions 0, 1, 2, 3, and 4 would two skipped alignments. Now for good character rule, the mismatch at the offset needs to be fixed with just two skipped alignments. However, since the pattern repeats every 3 bps, a mismatch at any position 3, 4, 5 would force a mismatch after 3 shifts. Thus, the mismatched base must be at offset 2. For there to be a mismatch that guarantees two skips, the letter must not be to the left of the pattern, and the only letter satisfying that requirement is A. Thus, we have ??ATGC as the required reference format to force two skips with both rules. This gives us 16 possible substrings since each ? can be 4 bases.